

Aprendizaje por Refuerzo

Fundamentos y Aplicaciones

Clase 1 — Introducción y Procesos de Decisión de Markov

Dr. Darío Ezequiel Díaz

5 de mayo de 2026

Facultad de Ingeniería — Posgrados
Universidad Austral

Bienvenida y contrato pedagógico

Panorama histórico

El RL frente a otros paradigmas

Procesos de Decisión de Markov

Cierre y trabajo para la próxima semana

Bienvenida y contrato pedagógico

Bienvenida al curso

- Curso de Actualización Continua de la Facultad de Ingeniería.
- **18 horas** distribuidas en **6 encuentros sincrónicos** de 3 horas.
- Modalidad a distancia: martes de 18 a 21 hs, del 5 de mayo al 9 de junio de 2026.
- Audiencia destinataria: profesionales con formación en estadística, matemática, ciencia de datos, ingeniería o disciplinas afines.

Premisa metodológica

Equilibrio sostenido entre **rigor formal** y **práctica computacional**, con perspectiva estadística transversal.

Dr. Darío Ezequiel Díaz

- Profesor de Posgrado — Facultad de Ingeniería, Universidad Austral.
- Líneas de trabajo: aprendizaje automático estadístico, modelización probabilística, optimización secuencial.
- Foro de consultas habilitado durante toda la cursada para acompañamiento continuo.

Filosofía del curso

Comprender *por qué* funcionan los algoritmos antes de aplicarlos. La intuición sin formalismo se quiebra; el formalismo sin intuición se olvida.

Estructura curricular — los seis encuentros

N°	Fecha	Contenido principal
1	5- may	Introducción al RL. Procesos de Decisión de Markov.
2	12- may	Políticas, funciones de valor, ecuación de Bellman, bandits.
3	19- may	Métodos Monte Carlo y Diferencias Temporales TD(0).
4	26- may	SARSA, Q-Learning, exploración ϵ -greedy.
5	2-jun	Aproximación de funciones, DQN, gradiente de política.
6	9-jun	Aplicaciones avanzadas, herramientas modernas, integración.

Cuatro unidades temáticas

Unidad 1 (encuentros 1–2): fundamentos. Unidad 2 (3–4): predicción.

Unidad 3 (4–5): control. Unidad 4 (5–6): RL profundo y aplicaciones.

Componentes formativos

- Exposiciones dialogadas con perspectiva probabilística.
- Notebooks Jupyter en **Python** y **R**.
- Estudios de caso en finanzas, salud, robótica.
- Foro permanente de discusión.

Esquema de evaluación

- Participación y foros: **20%**
- Ejercicios por unidad (3 entregas): **30%**
- Trabajo final integrador: **50%**

Aprobación: nota mínima 6/10 y asistencia $\geq 70\%$.

Tres canales centrales

- **Aula virtual** (Campus Virtual de Universidad Austral): material sincrónico, foros, entregas.
- **Repositorio GitHub**: notebooks reproducibles en Python y R, abribles directamente en Google Colab.
- **Carpeta bibliográfica** (Drive): textos de referencia curados.

Stack computacional

Python ≥ 3.10 · Jupyter · NumPy · Matplotlib · **Gymnasium**

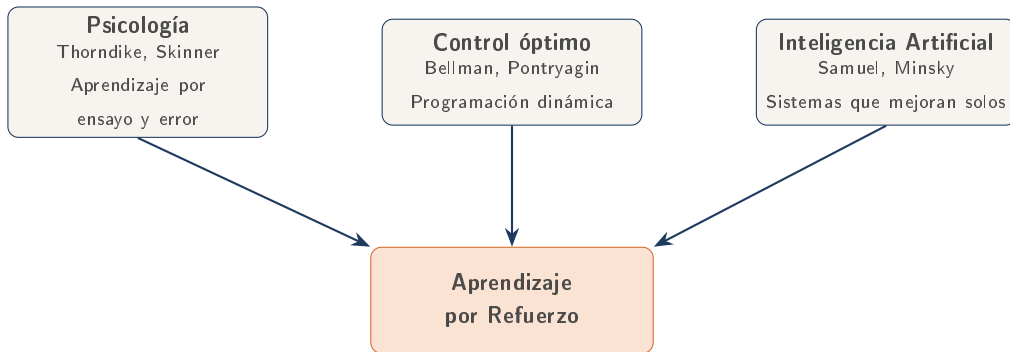
Para experimentación avanzada: PyTorch, Stable-Baselines3.

Versión R con reticulate (entorno conda rl-docencia).

Panorama histórico

¿De dónde viene el aprendizaje por refuerzo?

El RL no es una invención reciente: confluyen en él tres tradiciones intelectuales que durante décadas avanzaron por separado.



Las raíces psicológicas: la ley del efecto

Edward Thorndike (1898)

Estudió gatos atrapados en *cajas-problema*.
Observó que las acciones seguidas de consecuencias satisfactorias se repetían con mayor frecuencia.

Ley del efecto

Una acción cuyo resultado produce satisfacción incrementa su probabilidad futura de ocurrencia ante el mismo estímulo.

B. F. Skinner (1938)

Formalizó el **condicionamiento operante**: comportamientos modulados por reforzadores positivos y negativos.

Esta intuición — aprender a través de la consecuencia — constituye el corazón conceptual del RL contemporáneo.

Las raíces matemáticas: programación dinámica

Richard Bellman (1957) formalizó la noción de **decisión secuencial óptima** bajo incertidumbre.

Principio de optimalidad de Bellman

Una política óptima posee la propiedad de que, sea cual fuere el estado y la decisión inicial, las decisiones restantes deben constituir una política óptima respecto del estado resultante de la primera decisión.

Implicancia. La estructura recursiva del problema permite descomponerlo en subproblemas:

$$V(s) = \max_a \left\{ R(s, a) + \gamma \mathbb{E}[V(s') \mid s, a] \right\}$$

Esta ecuación, que estudiaremos en profundidad la próxima clase, es el **ADN matemático** del RL.

Década de 1980

Richard Sutton y Andrew Barto unifican las dos tradiciones, mostrando que el aprendizaje por ensayo y error puede formalizarse mediante programación dinámica aproximada.

Aportes seminales:

- Aprendizaje por **diferencias temporales** (TD).
- Algoritmos **actor-crítico** primigenios.
- Formulación canónica del problema RL.

Hito bibliográfico

Reinforcement Learning: An Introduction (Sutton & Barto, 1998; 2.^a ed. 2018) constituye la **biblia** del campo.

Texto principal de referencia para este curso.

Hitos contemporáneos

Año	Sistema	Significado
1992	TD-Gammon (Tesauro)	Backgammon a nivel campeón mundial mediante TD-learning con redes neuronales.
2013	DQN (DeepMind)	Atari desde píxeles: nace el <i>Deep RL</i> .
2016	AlphaGo	Derrota a Lee Sedol en Go, 10 años antes de lo previsto.
2017	AlphaZero	Aprende ajedrez, shogi y Go desde cero, sin datos humanos.
2019	OpenAI Five / AlphaStar	Dominio de juegos en tiempo real (Dota 2, StarCraft II).
2022	ChatGPT (RLHF)	RL aplicado al alineamiento de modelos de lenguaje.
2024	AlphaProof, AlphaFold-3	RL para descubrimiento matemático y científico.

Salud y medicina

- Dosificación adaptativa en sepsis y diabetes.
- Diseño de regímenes terapéuticos dinámicos.

Finanzas cuantitativas

- Optimización de carteras con horizonte largo.
- Estrategias de ejecución de órdenes (market impact).

Energía y procesos

- Refrigeración de centros de datos (Google).
- Despacho de redes eléctricas inteligentes.

Robótica y logística

- Manipulación con manos robóticas.
- Ruteo dinámico en cadenas de suministro.

Patrón unificador

Toma de decisiones **secuenciales** bajo incertidumbre con consecuencias **diferidas** y observabilidad **parcial**.

El RL frente a otros paradigmas

Los tres grandes paradigmas del aprendizaje automático

Supervisado

$$\{(x_i, y_i)\}_{i=1}^n$$

Etiquetas conocidas

Minimizar error de predicción

No supervisado

$$\{x_i\}_{i=1}^n$$

Sin etiquetas

Descubrir estructura latente

Por refuerzo

Trayectorias (s_t, a_t, r_t)

Recompensas *evaluativas*

Maximizar retorno acumulado

Cada paradigma responde a una pregunta diferente: *¿qué predigo?*, *¿qué descubro?*, *¿qué decido?*

Dado un dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ con y_i generado según una distribución condicional $P(y \mid x)$, se busca un estimador $\hat{f} : \mathcal{X} \rightarrow \mathcal{Y}$ que minimice el riesgo esperado:

$$\hat{f} = \arg \min_{f \in \mathcal{F}} \mathbb{E}_{(X, Y)}[\ell(f(X), Y)]$$

Características distintivas

- Existe una *respuesta correcta* y_i para cada entrada x_i .
- Las observaciones suelen ser **i.i.d.**
- Cada predicción se evalúa de forma **aislada**.
- Retroalimentación **instructiva**: indica qué se debió predecir.

Aprendizaje no supervisado — recordatorio

Dado $\mathcal{D} = \{x_i\}_{i=1}^n$ sin etiquetas, se busca **estructura latente**: agrupamientos, manifolds, factores generativos.

Bajo la formulación generativa, modelamos

$$p_{\theta}(x) = \int p_{\theta}(x, z) dz$$

con z variable latente no observada.

Características distintivas

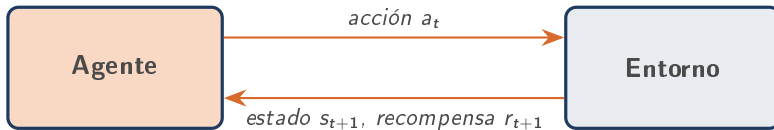
- No hay objetivo y externo.
- La evaluación recae en propiedades intrínsecas del modelo (verosimilitud, reconstrucción, separabilidad).
- La cantidad “a aprender” es *descriptiva*, no *prescriptiva*.

Lo que distingue al RL de los otros paradigmas

1. **Naturaleza secuencial.** Las decisiones afectan los datos futuros: la distribución de los estados depende de la política.
2. **Retroalimentación evaluativa.** La señal $r_t \in \mathbb{R}$ indica *qué tan buena* fue la acción, no *cuál hubiera sido la correcta*.
3. **Recompensas diferidas.** La consecuencia de una decisión puede manifestarse muchos pasos después (*credit assignment problem*).

Esta combinación produce un problema estadísticamente **no estándar**: las observaciones son correlacionadas, las distribuciones son no estacionarias y la inferencia debe combinar exploración con explotación.

El bucle agente-entorno



Protocolo de interacción — discretizado en el tiempo

En cada paso $t = 0, 1, 2, \dots$

1. El agente observa $s_t \in \mathcal{S}$.
2. Selecciona una acción $a_t \in \mathcal{A}$ según una política π .
3. El entorno retorna una recompensa r_{t+1} y un nuevo estado s_{t+1} .

	Supervisado	No supervisado	Por refuerzo
Datos	(x, y)	x	Trayectorias (s, a, r)
Señal	Etiqueta correcta	Sin señal externa	Recompensa escalar
Retroalimentación	Instructiva	—	Evaluativa, diferida
Distribución	Fija, i.i.d.	Fija, i.i.d.	Inducida por la política
Objetivo	Minimizar error	Modelar estructura	Maximizar retorno
Pregunta central	¿Qué predigo?	¿Qué descubro?	¿Qué decido?

Procesos de Decisión de Markov

¿Por qué necesitamos un lenguaje formal?

Los problemas que abordaremos comparten una estructura común:

- Un **estado del mundo** que cambia con el tiempo.
- Un **repertorio de acciones** disponibles para un agente.
- Una **dinámica estocástica** que conecta estados, acciones y consecuencias.
- Un **criterio** para juzgar trayectorias.

Necesidad metodológica

Antes de resolver un problema, debemos *formalizarlo*. El **Proceso de Decisión de Markov** (MDP) es el lenguaje canónico que adoptaremos a lo largo de todo el curso.

Definición formal de un MDP

Proceso de Decisión de Markov

Un MDP es una quintupla $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ donde:

- $\mathcal{S}$ es el **conjunto de estados**.
- $\mathcal{A}$ es el **conjunto de acciones**.
- $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ es la **función de transición**, con
$$P(s' \mid s, a) = \mathbb{P}(S_{t+1} = s' \mid S_t = s, A_t = a).$$
- $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ (o $\mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$) es la **función de recompensa**.
- $\gamma \in [0, 1]$ es el **factor de descuento**.

En el caso más general, R puede ser estocástica: $r_{t+1} \sim P_R(\cdot \mid s_t, a_t)$.

Representa toda la información necesaria para describir la situación actual del sistema.

Casos discretos finitos

- Posiciones en un tablero.
- Estados de un sistema de inventario.
- Casillas en GridWorld.

Casos continuos

- $(\theta, \dot{\theta})$ de un péndulo.
- Vector de precios en un mercado.
- Imagen de píxeles en Atari.

Observación clave

La elección de $\mathcal{S}$ es una **decisión de modelado** crítica. Estados pobres producen problemas no markovianos disfrazados; estados ricos explotan la maldición de la dimensionalidad.

El espacio de acciones $\mathcal{A}$

Conjunto de decisiones disponibles. Puede depender del estado: $\mathcal{A}(s) \subseteq \mathcal{A}$.

- **Discreto finito:** “arriba, abajo, izquierda, derecha”; “comprar, vender, mantener”.
- **Continuo:** torque aplicado a una articulación, fracción del capital invertida.
- **Estructurado:** secuencias, grafos, lenguaje natural (caso de RLHF en LLMs).

Implicancia algorítmica

La cardinalidad y estructura de $\mathcal{A}$ determinan qué familia de algoritmos resulta viable: tabulares, gradiente de política, métodos basados en valor con argmax, entre otros.

Captura la **estocasticidad del entorno**: dado un estado y una acción, ¿hacia dónde se mueve el sistema?

$$P(s' \mid s, a) = \mathbb{P}(S_{t+1} = s' \mid S_t = s, A_t = a)$$

Para todo (s, a) debe satisfacerse la condición de normalización:

$$\sum_{s' \in \mathcal{S}} P(s' \mid s, a) = 1 \quad (\text{caso discreto})$$

Distinción crucial: *model-based* vs. *model-free*

En la mayoría de aplicaciones reales, P es **desconocida**. El RL libre de modelo aprende directamente de la experiencia sin estimar P explícitamente.

La función de recompensa R

Codifica el **objetivo del agente**: qué configuraciones son deseables, cuáles deben evitarse.

$$r_{t+1} = R(s_t, a_t) \quad \text{o bien} \quad r_{t+1} \sim P_R(\cdot \mid s_t, a_t, s_{t+1})$$

Hipótesis de recompensa (Sutton & Barto)

Todo objetivo concebible puede formalizarse como la maximización del valor esperado de la suma acumulada de una señal escalar.

Diseño de recompensas (*reward shaping*): traducir un objetivo cualitativo en una señal numérica es, frecuentemente, la parte más artesanal del modelado RL. Recompensas mal diseñadas conducen a comportamientos no intencionados (*reward hacking*).

Pondera la importancia relativa de recompensas futuras frente a inmediatas, con $\gamma \in [0, 1]$.

Interpretaciones

- **Económica:** preferencia temporal (analogía con la tasa de descuento financiera).
- **Probabilística:** probabilidad de que el episodio continúe.
- **Matemática:** garantiza convergencia de la serie infinita.

Casos límite

- $\gamma = 0$: agente *miope*, sólo importa r_{t+1} .
- $\gamma \rightarrow 1$: horizonte largo, importan todas las recompensas futuras.
- Valores típicos en práctica:
 $\gamma \in \{0.9, 0.95, 0.99\}$.

La propiedad de Markov

El corazón conceptual del marco MDP.

Propiedad de Markov

$$\mathbb{P}(S_{t+1}, R_{t+1} \mid S_0, A_0, \dots, S_t, A_t) = \mathbb{P}(S_{t+1}, R_{t+1} \mid S_t, A_t)$$

En palabras: **el futuro es condicionalmente independiente del pasado dado el presente.**

Consecuencia operativa

El estado s_t debe contener *toda* la información relevante del pasado. Si la propiedad falla, debemos enriquecer la representación del estado o trabajar con observaciones parciales (POMDP).

Trayectoria

Una *trayectoria* (o *rollout*) es la secuencia generada por la interacción agente–entorno:

$$\tau = (S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \dots)$$

- **Tareas episódicas:** existe un estado terminal s_T . La trayectoria termina y comienza una nueva.
- **Tareas continuas:** no hay terminación natural; la trayectoria se prolonga indefinidamente.

La distribución de trayectorias depende conjuntamente de la dinámica P y de la política π del agente.

Retorno: la cantidad a optimizar

El **retorno** G_t a partir del paso t es la suma descontada de recompensas futuras:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Equivalentemente, en forma recursiva (núcleo de la futura ecuación de Bellman):

$$G_t = R_{t+1} + \gamma G_{t+1}$$

Objetivo del agente

Encontrar una política π que maximice el retorno esperado:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi}[G_0]$$

Horizonte finito vs. horizonte infinito

Horizonte finito T

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$$

- Tareas con final natural (juegos, ensayos clínicos).
- Política puede ser **no estacionaria**: π_t depende del paso.

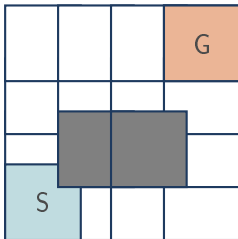
Horizonte infinito descontado

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \quad \gamma < 1$$

- Tareas continuas (control de procesos).
- Política **estacionaria**: $\pi(a | s)$ no depende de t .
- Convergencia garantizada si recompensas acotadas.

En este curso enfatizaremos el caso **descontado infinito**, marco teórico predominante en RL moderno.

Ejemplo canónico: GridWorld



Formalización como MDP

- $\mathcal{S} = \{(i, j) : i, j \in \{0, 1, 2, 3\}\} \setminus \text{muros}$
- $\mathcal{A} = \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$
- P : movimiento determinista o estocástico (e.g. resbalón).
- $R(s, a) = -1$ por paso, $+10$ al alcanzar G .
- $\gamma = 0.95$, episodio termina al llegar a G .

Por qué GridWorld

Sencillo de visualizar, conceptualmente rico, ideal para verificar algoritmos. Lo retomaremos en la Clase 2 y 4.

Anticipo: política y función de valor

Con el MDP formalizado, las preguntas centrales emergen naturalmente:

- **¿Cómo decide el agente?**

Mediante una **política** $\pi(a \mid s)$, distribución sobre acciones condicionada al estado.

- **¿Cómo se evalúa una política?**

A través de la **función de valor**:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

- **¿Cómo encontramos la mejor política?**

Resolviendo la **ecuación de Bellman**.

Próxima clase

Estos tres elementos — política, valor, ecuación de Bellman — estructurarán nuestro segundo encuentro.

**Cierre y trabajo para la próxima
semana**

- Políticas: deterministas, estocásticas, óptimas.
- Funciones de valor estado $V^\pi(s)$ y estado-acción $Q^\pi(s, a)$.
- **Ecuación de Bellman** y operador de Bellman: existencia y unicidad del punto fijo.
- Dilema exploración-explotación formalizado: el problema del **bandido multibrazo**.
- Estrategias: ε -greedy, UCB, Thompson sampling.
- Práctica: implementación de bandits en Python y R.

Configuración del entorno computacional

1. Instalar Python ≥ 3.10 (si no lo tiene).
2. Crear entorno virtual: `conda create -n rl-curso python=3.10`
3. Instalar dependencias: `pip install gymnasium numpy matplotlib jupyter`
4. Verificar funcionamiento con el notebook `Clase01_HelloRL_Python.ipynb`.

Para usuarios de R

Configurar `reticulate` con el entorno `conda rl-docencia` según la guía publicada en el aula virtual.

Notebooks disponibles en GitHub y Google Colab.

Bibliografía obligatoria de la Clase 1

- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2.^a ed.). MIT Press. **Capítulo 1 completo y Capítulo 3, secciones 3.1–3.4.**

Lectura recomendada

- Bertsekas, D. P. (2019). *Reinforcement Learning and Optimal Control*. Cap. 1.
- Murphy, K. P. (2024). *Probabilistic Machine Learning: Advanced Topics*. Sec. 35.1.

Profundización opcional

- Zhao, S. (2024). *Mathematical Foundations of Reinforcement Learning*. Cap. 1.7.

Gracias

Espacio para preguntas y discusión

Foro de Consultas habilitado en el aula virtual.